HP35 Calculator

Mostafa Faik

September 8, 2024

Introduction

In this assignment, we aimed to implement a simple calculator that processes mathematical expressions written in *reverse Polish notation (RPN). RPN* is a mathematical notation where every operator follows all of its operands. This type of notation eliminates the need for parentheses to indicate operation precedence. To achieve this, we made an extensive use of a stack, a fundamental data structure that follows a last-in, first-out (LIFO) principle.

The goal of this exercise was to gain a deeper understanding of stacks and how they can be applied to solve real-world problems, such as evaluating mathematical expressions. Additionally, we learned about two different types of stacks: `static stacks` and `dynamic stacks`, each with its own strengths and weaknesses.

Reverse Polish Notation

`Reverse Polish Notation (RPN)` is a mathematical notation where every operator follows all of its operands. This differs from the standard infix notation, where operators appear between operands (e.g., $3 + 5$). The main advantage of `RPN` is that it eliminates the need for parentheses. That is, the expression $(4 + 5) * 6$ in infix notation becomes $45 + 6*$ in `RPN`. The steps to evaluate an `RPN` expression involving a stack, is to:

- Push operands onto the stack.
- When an operator is encountered, pop the appropriate number of operands from the stack.
- Apply the operator to the popped operands.
- Push the result back onto the stack.

The Stack

We first implemented and tested two different stacks, `static stack` and `dynamic stack`. A stack is a data structure that allows two primary operations:

- Push: add an item to the top of the stack.
- Pop: remove the item from the top of the stack.

The stack pointer typically points to the top of the stack, which is the last element pushed onto the stack. When the stack is empty, the pointer can be initialized to -1, indicating no elements are present. This initialization is also important because it clearly distinguishes an empty stack (with a pointer at -1) from a stack with at least one element (with a pointer at 0 or higher). If the stack is full (i.e., the stack pointer is at its maximum allowed index), pushing a new value should trigger a `stack overflow` condition, while popping an item from an empty stack should trigger a `stack underflow` condition.

Static Stack

In this task, we implemented a `static stack`. A `static stack` has a fixed size that is determined at the time of creation. This means it can hold only a specific number of elements, and its size cannot be changed dynamically. This implementation is simple but can result in `stack overflow` if too many elements are pushed onto the stack. The following code shows how the `push()` and `pop ()` functions was implemented in a `static stack`:

```
public void push(int val) {
      if (top >= stack.length - 1) { // Check for stack overflow
          System.out.println("Stack overflow");
      } else {
          stack[++top] = val; // Increment top and push value onto the stack
      }
  }

  public int pop() {
      if (top < 0) { // Check for stack underflow
          System.out.println("Stack underflow");
          return -1; // Sentinel value for underflow
      } else {
          return stack[top--]; // Return top value and decrement top
      }
  }
```

The `static stack` implementation is a straightforward data structure that effectively manages integer values with simple push and pop operations. By carefully managing the stack pointer and checking for `overflow` and `underflow` conditions, the stack can handle basic stack operations with clear feedback on potential errors. This implementation serves as a foundation for more complex stack-based applications, such as an `RPN` calculator.

```
StaticStack s = new StaticStack(16);

        s.push(32);
        s.push(33);
        s.push(34);

        System.out.println("pop : " + s.pop());
        System.out.println("pop : " + s.pop());
        System.out.println("pop : " + s.pop());
        System.out.println("pop : " + s.pop());
```

When running the above code for the `static stack` function, the output was as follows:

```
pop : 34
pop : 33
pop : 32
Stack underflow
pop : -1
```

We observe from the output, that when we try to call the `pop()` function for the fourth time, we get a `stack underflow` indicating that the stack is empty. If we push more values than the stacks capacity, it will result in a `stack overflow` because a `static stack` has a fixed size.

Dynamic Stack

In this task, we implemented a `dynamic stack`, which is a stack data structure capable of growing and shrinking as needed. Unlike a `static stack`, which has a fixed size, a `dynamic stack` adjusts its capacity based on the number of elements it contains. This adjustment involves reallocating the stack to a larger or smaller array as needed.

This `dynamic stack` implementation provides a flexible and efficient way to manage a stack that adjusts its size based on usage patterns, ensuring optimal memory usage and performance. The `push ()` and `pop ()` functions was implemented as follows:

```
public void push(int val) {
        // Check if the stack is full
        if (top == size) {
            resize(size * 2); // Double the size if full
        }
        stack[top++] = val; // Push value and increment top
    }

    public int pop() {
        // Check for stack underflow
        if (top == 0) {
            System.out.println("Stack underflow");
            return -1; // Sentinel value for underflow
        } else {
            int value = stack[--top]; // Return top value and decrement top

            // Shrink the stack if necessary
            if (top > 0 && top == size / 4) {
                resize(size / 2);
            }

            return value;
        }
    }
```

We can observe from the code, that the `dynamic stack` is an extension of the `static stack`, but instead of throwing exception for every `stack overflow`, we create an array of twice the size of the original array with the help of a `resize ()` function. This helps to allocate more memory to all the new values that have been pushed onto the stack.

This shows, that the `dynamic stack` efficiently manages memory by resizing itself based on the number of elements. It grows by doubling its size when necessary and shrinks by halving its size only when the stack is significantly under-utilized. This strategy minimizes the frequency of resizing operations, balancing memory usage and performance.

Implementing the HP35 Calculator

The final task was to implement the HP35 calculator with the stack in place. The calculator was implemented using the `reverse Polish notation (RPN)` and some skeleton code that was provided in the task.

```
switch (input) {
    case "+":
```

```
if (stack.getSize() < 2) {
    System.out.println("Insufficient values in the stack");
} else {
    int b = stack.pop();
    int a = stack.pop();
    stack.push(a + b);
}
break;

case "-":
if (stack.getSize() < 2) {
    System.out.println("Insufficient values in the stack");
} else {
    int b = stack.pop();
    int a = stack.pop();
    stack.push(a - b);
}
break;

case "*":
if (stack.getSize() < 2) {
    System.out.println("Insufficient values in the stack");
} else {
    int b = stack.pop();
    int a = stack.pop();
    stack.push(a * b);
}
break;

case "/":
if (stack.getSize() < 2) {
    System.out.println("Insufficient values in the stack");
} else {
    int b = stack.pop();
    int a = stack.pop();
    if (b == 0) {
        System.out.println("Division by zero");
        stack.push(a); // Push back the values
        stack.push(b);
    } else {
        stack.push(a / b);
    }
}
break;
```

We can observe, that the calculator can perform arithmetic operations by popping values from the stack, pushing the result back onto the stack and read input from the terminal. When running the code and writing the following expression:

$$423 * 4 + 4 * +2-$$

the result we get is 42, because according to the `reverse Polish notation` we:

- Push 4, 2, and 3 onto the stack.
- Pop 2 and 3, then multiply them: 2 * 3 = 6.
- Push the result 6 back onto the stack.
- Push 4 onto the stack: [4, 6, 4].
- Pop 6 and 4, then add them: 6 + 4 = 10.
- Push the result 10 back onto the stack.
- Push 4 onto the stack: [4, 10, 4].
- Pop 10 and 4, then multiply them: 10 * 4 = 40.
- Push the result 40 back onto the stack.
- Pop 4 and 40, then add them: 40 + 4 = 44.
- Push the result 44 back onto the stack.
- Push 2 onto the stack: [44, 2].
- Pop 44 and 2, then subtract them: 44 - 2 = 42.

Conclusion

In this assignment, we explored the use of `reverse Polish notation` and how it simplifies the evaluation of mathematical expressions by eliminating the need for parentheses. We also implemented a `static stack` and a `dynamic stack`, both of which were used to develop a basic calculator that evaluates `RPN` expressions. Through this assignment, we have gained practical experience in using stacks and learned about the trade-offs between static and dynamic data structures. The `dynamic stack` provides flexibility in terms of size management, while the `static stack` is more straightforward but limited in capacity.